Chapitre 3

Calculabilité, décidabilité.

Sommaire.

1 P et NP 1

2 Décidabilité. 4

Les propositions marquées de ★ sont au programme de colles.

1 P et NP

Définition 1

P et NP sont des classes de complexité. Les classes de complexité sont des ensembles de problèmes que l'on rassemble en fonction de leur difficulté selon plusieurs critères.
La classe P correspond aux problèmes de décision qui peuvent être résolus en temps polynomial.
La classe NP correspond aux problèmes pour lesquels on peut vérifier une solution en temps polynomial.

Définition 2

Une instance I d'un problème P est dite positive si la réponse à la question est `Oui`.
Un algorithme V est un vérificateur de P si I positif $\iff \exists c$ tel que $V(I,c) = \texttt{Oui}$.

Exemple 3

Étant donné G un graphe et k un entier, est-il possible de colorer S avec les k premiers entiers en évitant d'avoirs deux voisins de même couleur ?
Un vérificateur : pour tout sommet on vérifie si ses voisins sont de couleur différente.

Définition 4

Un problème A appartient à NP si :
— Il existe un vérificateur V qui s'exécutre en temps polynomial en la taille de ses entrées.
— Il existe un polynome R tel que la taille d'un certificat est bornée par $R(|I|)$.

Exemple 5: NP ?

Problème A : étant donnés un graphe G, un entier k, existe-t-il un sous-graphe G' à k sommets tels qu'ils soient tous voisons deux-à-deux ?
Problème B : étant donnés une formule φ, est-elle satisfiable.

Théorème 6: $\texttt{P} \subset \texttt{NP}$

Soit $P_1 \in \texttt{P}$. Il existe A un algorithme polynomial pour le résoudre.
On construit V_p prenant une instance I de P, un certificat C et renvoyant $A(I)$.
Alors V_p s'exécute bien en temps polynomial, et c'est bien un vérificateur.
I positive $\iff A(I) = \texttt{Oui} \iff \exists c,\ V_p(I,c) = \texttt{Oui}$.
On a $R = 1$ convient puisqu'on peut choisir $c = \varepsilon$, alors $\texttt{P}_1 \in \texttt{NP}$ donc $\texttt{P} \subset \texttt{NP}$.

Définition 7: Réduction.

Une réduction de P_1 vers P_2 est une fonction f qui à une instance de P_1 associe une instance de P_2 et :
— I est une instance positive de P_1 ssi $f(I)$ est une instance positive de P_2.

On dit d'une réduction qu'elle est polynomiale si elle se calcule en temps polynomiale en la taille de son entrée.

Exemple 8

Ensemble Indépendant :
— Instance : graphe G, entier k.
Existe-t-il $S' \subset S$ de taille k tel que $\forall s,t \in S',\ (s,t) \notin A$.

Couverture par les sommets :
— Instance : graphe G, entier k.
Existe-t-il $S'' \subset S$ tel que $\forall (x,y) \in A,\ x \in S''$ ou $y \in S''$ et $|S''| = k$.

Ensemble Indépendant se réduit polynomialement au problème Couverture par les sommets.

Solution :

$\boxed{\Longrightarrow}$ Soit $f : (G,k) \mapsto (G, |S| - k)$. Montrons que f est une réduction.
Soit (G,k) positive de Ensemble indépendant. Il existe S' qui convient.
Alors $\forall s,t \in S',\ (s,t) \notin A$, $\forall (s,t) \in A,\ s \notin S'$ ou $t \notin S'$.
Donc $\forall (s,t) \in A,\ s \in S \setminus S'$ ou $t \in S \setminus S'$.
$|S \setminus S'| = |S| - k$. Donc $|S''| = S \setminus S'$ convient.
$\boxed{\Longleftarrow}$ Soit (G,k) instance de Ensemble indépendant telle que $f(G,k)$ est positive de Couverture par les sommets.
Il existe S'' de taille $|S| - k$ tel que $\forall (x,y) \in A,\ x \in S''$ ou $y \in S''$.
$\forall s,t \notin S'',\ (s,t) \notin A$, $\forall s,t \in S \setminus S'',\ (s,t) \notin A$. Donc $S \setminus S''$ est un ensemble indépendant de taille k de G, (G,k) est une instance positive de Ensemble Indépendant. La réduction est polynomiale. Ensemble indépendant se réduit polynomialement à Couverture par les sommets.

Proposition 9

Soit f une réduction polynomiale de P_1 vers P_2 et g de P_2 vers P_3.
Alors $g \circ f$ est une réduction polynomiale de P_1 vers P_3.

Preuve :

I instance positive de $P_1 \iff f(I)$ instance positive de $P_2 \iff g \circ f(I)$ instance positive de P_3.
On calcule $f(I)$ en temps polynomial en taille de I, $g \circ f(I)$ en temps polynomiale en taille de $f(I)$.

Proposition 10

Si $f(P_1) \subset P_2$, et que P_2 est de classe `P`, alors P_1 l'est aussi.

Preuve :

Il existe A un algorithme résolvant P_2 en temps polynomial. Il existe f une réduction polynomiale.
On pose l'algorithme B renvoyant $A(f(I))$.
Alors I positif $\iff f(I)$ positif $\iff A(f(I))$ vrai $\iff B(I)$ vrai.
Donc B résout P_1 en temps polynomial. Donc P_1 est de classe `P`.

Proposition 11

Si P_1 se réduit polynomialement à P_2, alors $P_2 \in \texttt{NP} \implies P_1 \in \texttt{NP}$.

Preuve :

Il existe un vérificateur V et un polynôme R qui conviennent et une réduction polynomiale f de P_1 vers P_2.
Vérificateur V' : instance I, certificat $c \mapsto V(f(I), c)$.
I positive pour $P_1 \iff f(I)$ est positive pour $P_2 \iff \exists c,\ |c| \leq R(|f(I)|)$ et $V(f(I), c)$ vrai
$\iff \exists c,\ |c| \leq R(|f(I)|)$ et $V'(I, c)$ vrai $\iff \exists c,\ |c| \leq R'(|I|)$ et $V'(I, c)$ vrai.
Donc $P_1 \in \texttt{NP}$.

Définition 12

Un problème P est dit `NP`-difficile si $\forall P' \in \texttt{NP},\ P'$ se réduit polynomialement à P.
Un problème P est dit `NP`-complet si il est `NP`-difficile et dans `NP`.

Proposition 13

Si P_1 est `NP`-complet et se réduit polynomialement à P_2, alors P_2 est `NP`-difficile.

Preuve :

Soit $P' \in \texttt{NP}$. Il se réduit polynomialement à P_1, il se réduit polynomialement à P_2, donc P_2 est `NP`-difficile.

Théorème 14: de Cook. (Admis)

Le problème SAT est `NP`-difficile, il est donc `NP`-complet.

Exemple 15: 3-SAT

Le problème 3-SAT est `NP`-complet.

Solution :

Il est dans `NP`, même vérificateur que pour 2-SAT.
Soit φ une instance de SAT-FNC, conjonction de n clauses c_i.
Pour chaque c_i, on modifie en fonction de sa forme :
— $c_i = l_j \to l_j \vee l_j \vee l_j$.
— $c_i = l_j \vee l_k \to l_j \vee l_k \vee l_j$.
— $c_i = l_j \vee l_k \vee l_l \to l_j \vee l_k \vee l_k$.
— $c_i = l_{i_1} \vee ... \vee l_{i_n} \to (l_{i_1} \vee l_{i_2} \vee y_{i_1}) \wedge (\neg y_{i_1} \vee l_{i_3} \vee y_{i_4}) ... \wedge (\neg y_{i(n-3)} \vee l_{i(n-1)} \vee l_{i_n})$.
On a défini f, on pose donc φ' la conjonction des $f(c_i)$.
$\boxed{\implies}$ Supposons φ satisfiable. Il existe v valuation telle que $v(\varphi) = \top$.
Alors $\forall i \in [\![1, n]\!],\ \exists l_i \in c_i,\ v(l_i) = \top$, si $|c_i| \leq 3$, alors $v(f(c_i)) = \top$.
Si $|c_i| \geq 4$, il existe l_{ij} tel que $v(l_{ij}) = \top$.
— Si $j \leq 2$, on fixe $v'(y_i k) = \bot$ pour tout $k \in [\![1, n-3]\!],\ v'(f(c_i)) = \top$.
— Si $j \geq n-1$, $\forall k \in [\![1, j-2]\!]$, $v(y_{ik}) = \top$, les autres à $\bot$.
Pour chaque clause, on peut étendre v en v' satisfaisant l'image par f, donc $v'(\varphi') = \top$.
$\boxed{\impliedby}$ Soit φ' satisfiable, $\exists v',\ v'(\varphi') = \top$.
On considère v : v' restreinte aux variables de φ. Montrons $v(\varphi) = \top$.
— c_i est de taille ≤ 3 : $v'(f(c_i)) = \top \iff v(c_i) = \top$.
— $v'(f(c_i)) = \top$ sinon.
Il y a $n-2$ clauses dans $f(c_i)$ et $n-3$ nouvelles variables. Chaque variable satisfait au plus une clause, il y a une clause qui est satisfiable par un littéral :
— $v'(l_{ik}) = \top \implies v(l_{ik}) = \top \implies v(c_i) = \top$.
Donc $\forall i,\ v(c_i) = \top \implies v(\varphi) = \top$, donc φ satisfiable.
La fonction est bien une réduction de SAT-FNC à 3-SAT.

La réduction est bien polynomiale : dans le cas d'une clause de taille supérieure à 4, on crée $n-2$ clauses de taille 3, chacune en temps polynomial.

Exemple 16

Montrer que Clique est NP-complet.

Solution :

Soit une formule sous FNC. On fait le graphe des littéraux, avec des arêtes ssi ils ne sont pas dans la même clause.
On cherche une clique dans le graphe obtenu, elle sera de taille au plus K, le nombre de clauses. Montrons que c'est une réduction.
$\boxed{\Longrightarrow}$ Supposons φ satisfiable. Il existe v tel que $v(\varphi) = \top$. Pour chaque clause c_i, il y a au moins un l_{ik} tel que $v(l_{ik}) = \top$. On peut choisir un tel littéral pour chaque clause pour former un ensemble S' de sommets.
Alors $\forall l_{ik}, l_{jk} \in S',\ l_{ik} \neq l_{jk} \Longrightarrow i \neq j$ et $v(l_{ik}) = v(l_{jk}) = \top$.
Ils ne sont pas négation l'un de l'autre et appartiennent à des clauses différentes, donc $(l_{ik}, l_{jk}) \in A$ et $|S'| = K$.
$\boxed{\Longleftarrow}$ Supposons que G admette une clique de taille K : $\exists S' \subset S,\ |S'| = K$ et $\forall s, t \in S',\ s \neq t \Longrightarrow (s, t) \in A$. On fixe v une valuation telle que $\forall l_{ik} \in S',\ v(l_{ik}) = \top$, bien définie car S' ne contient pas un élément et sa négation.
Chaque clause a au plus un élément dans S', puisque $|S'| = K$, chaque clause a exactement un élément dans S'.
Donc chaque clause possède un élément évalué à $\top$ par v, donc chaque clause est évaluée à $\top$, donc $v(\varphi) = \top$.

C'est une réduction; elle est bien polynomiale car pour n clauses, on a $3n$ sommets et au plus $9n(n-3)$ arêtes.

Exemple 17

Cycle Hamiltonien :
Instance : Graphe G.
Question : Existe-t-il un cycle qui passe une unique fois par chaque sommet ?

Voyageur de commerce :
Instance : Graphe complet pondéré G, un entier k.
Question : Existe-t-il un cycle qui passe une unique fois par chaque sommet de poids total inférieur à k ?
On admet Cycle Hamiltonien NP-difficile, montrons que voyageur de commerce NP-complet.

Solution :

Voyageur de commerce est NP. Vérificateur : instance I et liste de sommets : Tient une liste des sommets vus, et la somme des poids des arêtes vues. Renvoie Vrai si Vus ne contient pas faux et somme de poids inférieur à k.

Réduction : il existe un cycle qui passe exactement une fois par tous les sommets, on fixe b le poids des arêtes bleues à 1, et celui des rouges à $|S|+1$. Un tel cycle dans un tel graphe ne passe que par des arêtes bleues et une fois par chaque sommet a un poids $|S|$. Si le cycle emprunte une arête rouge, son poids est d'au moins $|S|+1$. On prend donc $K = |S|$. On a construit une réduction. En effet, si G contenait un cycle hamiltonien, son image contient un cycle hamiltonien composé d'arêtes bleues, un cycle de poids total inférieur à K.
Si l'image contient un cycle hamiltonien de poids total inférieur à K, il n'est composé que d'arêtes bleues et était donc présent dans G.

Exemple 18: Sac à dos.

Le problème appartient à la classe NP, il est NP-complet.
On étudie un algorithme de programmation dynamique. On étudie $V_{i,p}$ la valeur maximale que l'on peut transporter en n'utilisant que les i premiers objets et avec un poids total inférieur ou égal à p.
On a alors $V_{i+1,p} = \max(V_{i,p}, V_{i,p-p_{i+1}} + v_{i+1})$, $V_{0,p} = 0$.

Algorithme 1 : Algorithme de programmation dynamique sac à dos.

Entrées : Deux tableaux V_i et P_i, un entier P et un entier V.
$T \leftarrow$ Tableau de taille $(n+1) \times (p+1)$.
pour p allant de 0 à P **faire**
  $T[0][p] = 0$.
fin
pour i allant de 1 à n **faire**
  pour p allant de 0 à $p_i - 1$ **faire**
    $T[i][p] = T[i-1][p]$.
  fin
  pour p allant de p_i à P **faire**
    $T[i][p] = \max(T[i-1][p], T[i-1][p-P_i] + v_i)$.
  fin
fin
retourner $T[n][p] \geq v$.

Ce programme s'exécute en $O(nP)$ et décide bien si une instance de sac-à-dos est positive.
On a l'impression d'avoir une solution polynomiale d'un algorithme NP-complet, c'est une illusion.
En effet, P est de taille $O(\log P)$, donc la taille de l'instance : $O(\sum(\log v_i + \log p_i) + \log P + \log V)$.
Donc $P = 2^{\log P}$, donc nP n'est pas polynomial en la taille de l'instance. Notre algorithme ne prouve pas que sac-à-dos appartient à P.

Exemple 19: Bin Packing

Instance : n nombres rationnels p_i/q_i, un entier m.
Question : Existe-t-il une partition des n nombres en m ensembles B_j tels que $\forall j,\ \sum_{B_j} \frac{p_i}{q_i} \leq 1$?
Vérificateur : Entrée : Instance, On crée L une liste composée de $(j, \sum_{i \in B_j} \frac{p_i}{q_i})$. Pour i 1 à n, si $(T[i], x)$ est dans L, le remplacer par $(T[i], x + \frac{p_i}{q_i})$.
Sinon, ajouter $(T[i], \frac{p_i}{q_i})$. Renvoyer si tous les couples de L ont un second élément ≤ 1.
Toutes les opérations sur celles-ci sont donc réalisées en temps polynomial en n.

Exemple 20: Partitionnement Carré

Instance : n entiers t_i, un entier N.
Question : Est-il possible de partitionner le carré de taille $N \times N$ en n carrés de tailles $t_i \times t_i$?
Vérificateur : Instance, Certificat : position des carrés à placer.
On vérifie qu'il n'y a pas de superpositions, de débordements et que la somme des aires est égale à N^2.

Variante : n couples (t_i, e_i), un entier N.
Question : Est-il possible de partitionner le carré de taille $N \times N$ avec e_i carrés de taille $t_i \times t_i$.
La taille de l'instance est beaucoup plus compacte; ce qui rend l'ancien vérificateur non polynomial.

2 Décidabilité.

Définition 21

On dit qu'un problème de décision est décidable s'il existe un algorithme qui s'arrête sur toutes ses instances et renvoie Vrai ssi elle est positive.

Théorème 22: Machine universelle.

Il existe un algorithme, dit **machine universelle**, qui étant donné une fonction f et une entrée x, simule $f(x)$.

Exemple 23: Problème de l'Arrêt.

Instance : Une fonction f, un paramètre x.
Question : La fonction f se termine-t-elle sur l'entrée x ?
Ce problème est indécidable.

Solution :

Supposons qu'un tel algorithme existe.
On définit `Paradoxe` tel que :
— Si `Arrêt`(f, f), alors boucle infinie.
— Sinon, renvoyer vrai.
Alors `Paradoxe(Paradoxe)` s'arrête ssi `Arrêt(Paradoxe, Paradoxe)=Faux` par définition, c'est absurde.

Proposition 24

S'il existe une réduction de P_1 à P_2 et que P_2 est décidable, alors P_1 l'est aussi.